# Parametric Weapon Design Optimisation: A Comparison of Spline Profile Families with FEA-in-Loop Evaluation

[Author]

February 26, 2026


## Abstract

We present a modular optimisation framework for the automated design of combat robot spinning weapons. The system exposes five independent *swap points*—Profile, Cutout, Objective, Optimizer, and FEA—that can be configured independently via a single parameter dataclass. Three spline profile families are compared: a periodic cubic B-spline ($C^2$, global support), a composite cubic Bézier ($C^1$, local support), and a centripetal Catmull-Rom spline ($C^1$, interpolating). A frozen Fourier radial series serves as the baseline. Weight-reduction pockets are modelled as superelliptic cutouts parameterised in polar coordinates. The enhanced objective replaces formula-constant bite depth with a kinematic Archimedean spiral simulation and replaces a geometric structural proxy with coarse-mesh 2D plane-stress FEA evaluated inside the optimisation loop. Differential evolution drives a two-phase search over $N$=12 radial control points (Phase 1) and up to six polar cutout parameters (Phase 2). Preliminary results on four weapon archetypes show that the B-spline family converges most reliably, while centripetal Catmull-Rom yields the most interpretable parameter space.


## 1 Introduction

Combat robot competitions demand spinning weapons that maximise stored kinetic energy, deliver large bite depth per tooth engagement, survive centrifugal and impact stresses, and remain within strict mass and envelope rules. Manual design of these weapons is iterative and heavily experience-dependent; even expert builders rely on trial-and-error tuning of shape parameters whose interactions are poorly understood.

Automated optimisation of weapon geometry has the potential to explore the full design space systematically and to make engineering trade-offs explicit. However, the utility of any optimisation study depends critically on the fidelity of the models used: a bite-depth formula that is constant for a given RPM and style cannot reward geometry that genuinely increases engagement, and a geometric structural proxy cannot detect stress concentrations introduced by weight-relief pockets.

This paper describes a research comparison framework built around five independently selectable *swap points*.

Table 1: Swap-point summary (see also `docs/ARCHITECTURE.md`).

| Slot | Interface | Implementations | Config |
|---|---|---|---|
| Profile | radii → poly | fourier; bspline; bezier; catmull | profile_type |
| Cutout / P2 | poly → poly+holes | fourier ellipses; superellipse pockets; topology (SIMP) | cutout_type |
| Objective | poly → metrics | baseline (formula + geometric proxy); enhanced (spiral bite + FEA) | evaluation_mode |
| Optimizer | cfg → run | baseline DE; enhanced DE | evaluation_mode |
| FEA | poly → fea score | off; coarse; fine; spiral contact loads (post-hoc) | fea_interval |

The *baseline* stack (frozen, unmodified) uses a Fourier radial profile, elliptic Fourier cutouts, a formula-constant bite model, and a geometric structural proxy—mirroring prior practice. The *enhanced* stack replaces each component in turn: a periodic B-spline (or Bézier, or Catmull-Rom) outer profile, polar superelliptic cutouts, a kinematic spiral bite simulation, and coarse-mesh FEA evaluated at every objective call. All results are compared side-by-side via `evaluate.py --mode compare`.

## 2 System Architecture

### 2.1 Swap Points

The optimisation stack is built from five independently configurable components, each governed by a field in the `OptimizationParams` dataclass. Table 1 lists the swap points, their interface contracts, the available implementations, and the controlling config key.

### 2.2 Data Flow

A single enhanced-mode evaluation follows the pipeline in Figure 1. The `WeaponConfig` is the single root object; every component reads only the fields it needs.

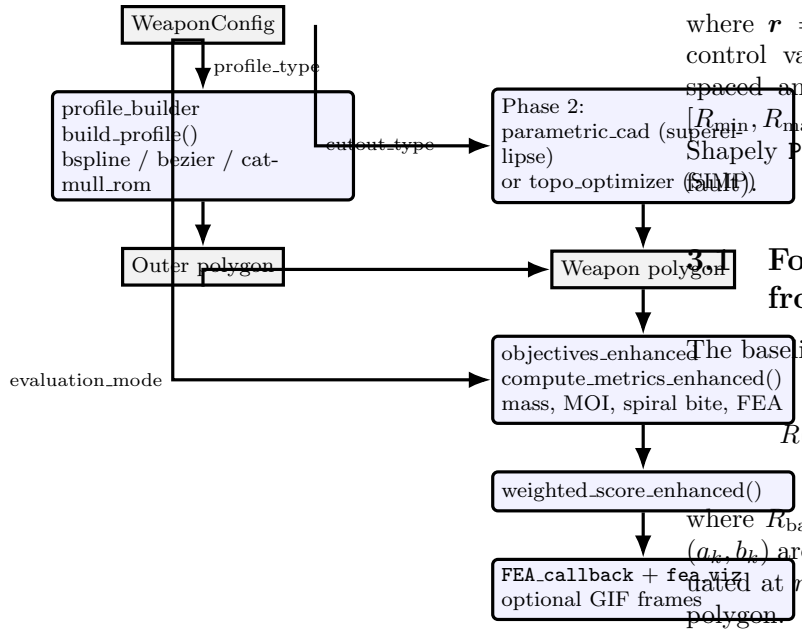


Figure 1: Enhanced-mode data flow (WeaponConfig driven).

Recent code updates (see `ARCHITECTURE.md`) add two notable options not present in the original draft: (i) `cutout_type = topology`, which replaces Phase 2 DE pockets with a SIMP topology optimisation loop; and (ii) optional spiral contact force injection into FEA for post-hoc stress studies. The data-flow diagram above already includes both paths.

### 2.3 Permutation Matrix

Table 2 lists the valid slot combinations. The first row is the frozen baseline used as the research reference; all other rows are enhanced variants explored in this work.

Table 2: Valid slot combinations (see `docs/ARCHITECTURE.md`).

| Profile | Cutout | Obj. | FEA | Description |
|---|---|---|---|---|
| fourier | fourier | baseline | off | Frozen baseline |
| bspline | superellipse | enhanced | coarse | Enhanced default |
| bspline | topology | enhanced | coarse | SIMP Phase 2 |
| bezier | superellipse | enhanced | coarse | Bézier variant |
| catmull_rom | superellipse | enhanced | coarse | Catmull-Rom variant |
| any spline | superellipse or topology | enhanced | fine | Hi-res FEA/GIF |
| bspline | fourier | enhanced | off | Legacy cutouts |

## 3 Profile Parametrisations

All four profile families share the same interface:

$$f(\boldsymbol{r},\, R_{\max},\, R_{\min},\, n_{\text{eval}}) \longrightarrow \text{Polygon} \mid \texttt{None} \tag{1}$$

where $\boldsymbol{r} = [r_0, r_1, \ldots, r_{N-1}]$ is the vector of $N$ radial control values in millimetres, one at each uniformly-spaced angle $\theta_i = 2\pi i/N$. All radii are clamped to $[R_{\min}, R_{\max}]$ before evaluation. The output is a closed Shapely `Polygon` with $n_{\text{eval}}$ vertices ($n_{\text{eval}} = 360$ by default).

### 3.1 Fourier Radial Series (baseline, frozen)

The baseline profile is a Fourier radial series:

$$R(\theta) = R_{\text{base}} + \sum_{k=1}^{K} \big(a_k \cos k\theta + b_k \sin k\theta\big) \tag{2}$$

where $R_{\text{base}}$ is the mean radius and the $2K$ coefficients $(a_k, b_k)$ are the optimisation variables. The profile is evaluated at $n_{\text{eval}}$ equally-spaced angles and wrapped into a polygon.

The Fourier series is implemented in `parametric.py` and is frozen for the duration of this study. Its main limitation is that every coefficient affects the entire profile (*global support*), so small perturbations to one coefficient can propagate oscillations around the whole weapon. Additionally, abrupt changes between optimiser steps can excite high-frequency Gibbs-type ringing when $K$ is moderate and the target profile has sharp features.

### 3.2 Periodic Cubic B-Spline

The B-spline profile replaces the frequency-domain parameters with $N$ radial control values at uniformly-spaced angles:

$$\theta_i = \frac{2\pi i}{N}, \quad i = 0, \ldots, N-1 \tag{3}$$

Control point $i$ is placed at $(r_i \cos\theta_i,\ r_i \sin\theta_i)$. A periodic cubic B-spline is then fitted through these $N$ Cartesian control points using `scipy.interpolate.splprep` with `per=True` and `k=3` (cubic, $s$=0 interpolating). The spline is evaluated at $n_{\text{eval}}$ uniformly-spaced parameter values $u \in [0, 1)$ to produce the polygon vertices.

**Properties.** The periodic B-spline satisfies $C^2$ continuity everywhere by construction. Because the underlying B-spline basis has *local support* of width $4/N$ in parameter space, moving a single control radius $r_i$ affects the profile only in the angular neighbourhood $[\theta_i - 4\pi/N,\ \theta_i + 4\pi/N]$, not the full weapon outline. No Gibbs phenomenon occurs because the spline degree is fixed at three regardless of $N$. The control radii are directly interpretable: $r_i$ is (approximately) the weapon radius at angle $\theta_i$.

### 3.3 Composite Cubic Bézier

The composite Bézier profile uses $N$ cubic Bézier segments joined at the $N$ control points, yielding a closed

curve with $C^1$ continuity at each join. Tangents are computed via Catmull-Rom-style central differences:

$$\boldsymbol{T}_i = \tfrac{1}{2}(\boldsymbol{P}_{i+1} - \boldsymbol{P}_{i-1}) \tag{4}$$

with periodic (modular) index arithmetic. For segment $i$ from $\boldsymbol{P}_i$ to $\boldsymbol{P}_{i+1}$, the four Bézier control points are:

$$\boldsymbol{C}_{1,i} = \boldsymbol{P}_i + \tfrac{1}{3}\boldsymbol{T}_i \tag{5}$$

$$\boldsymbol{C}_{2,i} = \boldsymbol{P}_{i+1} - \tfrac{1}{3}\boldsymbol{T}_{i+1} \tag{6}$$

Each segment is evaluated using the standard cubic Bézier formula:

$$\boldsymbol{B}(t) = (1-t)^3\boldsymbol{P}_i + 3(1-t)^2 t\,\boldsymbol{C}_{1,i} + 3(1-t)t^2\boldsymbol{C}_{2,i} + t^3\boldsymbol{P}_{i+1} \tag{7}$$

**Properties.** $C^1$ continuity (matched tangents, possible curvature discontinuity at joins). Each segment is controlled by exactly two adjacent control points plus their neighbours—tighter local support than the B-spline. The parameter space is identical to the B-spline: $N$ radii at equally-spaced angles.

### 3.4 Centripetal Catmull-Rom

The centripetal Catmull-Rom profile uses the Barry-Goldman recursive algorithm to evaluate a closed Catmull-Rom spline with parameterisation exponent $\alpha = 0.5$. For each segment $i$, four control points are used: $(\boldsymbol{P}_{i-1}, \boldsymbol{P}_i, \boldsymbol{P}_{i+1}, \boldsymbol{P}_{i+2})$ with periodic wrapping.

The centripetal knot sequence is:

$$t_0 = 0, \quad t_{k+1} = t_k + \|\boldsymbol{P}_{k+1} - \boldsymbol{P}_k\|^\alpha \tag{8}$$

The local parameter $t \in [0, 1]$ is remapped to $[t_1, t_2]$ for each segment. The Barry-Goldman algorithm computes the Catmull-Rom point via two levels of linear interpolation:

$$A_j = \mathrm{lerp}(\boldsymbol{P}_{j-1}, \boldsymbol{P}_j;\ t_{j-1}, t_j,\ t) \tag{9}$$

$$B_j = \mathrm{lerp}(A_j, A_{j+1};\ t_{j-1}, t_{j+1},\ t) \tag{10}$$

$$\boldsymbol{C}(t) = \mathrm{lerp}(B_1, B_2;\ t_1, t_2,\ t) \tag{11}$$

**Properties.** Unlike the B-spline and Bézier families, the Catmull-Rom spline *interpolates* every control point: the curve passes exactly through $(r_i \cos\theta_i,\ \ r_i \sin\theta_i)$ for all $i$. This makes the parameter space maximally interpretable—$r_i$ is the exact weapon radius at angle $\theta_i$. The centripetal variant ($\alpha$=0.5) spaces knots proportionally to $\sqrt{\|\Delta\boldsymbol{P}\|}$, preventing cusps and self-intersections that the uniform variant ($\alpha$=0) can produce for widely-spaced or non-uniform control points. Continuity is $C^1$.

### 3.5 Profile Family Comparison

Table 3 summarises the key properties of each family.

Table 3: Profile parametrisation comparison.

| Family | Cont. | Local | Interp. | Gibbs |
|---|---|---|---|---|
| Fourier | $C^\infty$ | No | No | Yes |
| B-spline | $C^2$ | Yes | No | No |
| Bezier | $C^1$ | Yes | No | No |
| Catmull-Rom | $C^1$ | Yes | Yes | No |

Cont. = continuity order; Local = local support; Interp. = interpolates control p

## 4 Evaluation Methods

### 4.1 Baseline Objective

The baseline objective, implemented in `objectives.py`, computes a weighted sum of five sub-scores:

$$S_{\text{base}} = w_1 E + w_2 B + w_3 IZ + w_4 Str + w_5 Sym \tag{12}$$

where $E$ is stored kinetic energy (from `physics.stored_energy_joules`), $B$ is bite depth from the formula model, $IZ$ is the impact zone score, $Str$ is the structural score, and $Sym$ is a symmetry penalty.

**Formula bite.** Bite depth in the baseline is computed by `bite_mm()` in `physics.py`:

$$b = \frac{v_{\text{approach}}}{n_{\text{teeth}} \cdot f_{\text{rot}}} \tag{13}$$

where $v_{\text{approach}}$ is the closing speed (m/s), $n_{\text{teeth}}$ is a *hard-coded* integer per weapon style (1, 2, or 3), and $f_{\text{rot}}$ is the rotation frequency (Hz). Because $n_{\text{teeth}}$ is not derived from the weapon geometry, the optimiser cannot increase the score by modifying the number or prominence of teeth.

**Geometric structural proxy.** The structural score in `structural.py` uses minimum wall thickness and the section width at the bore as a proxy for bending stiffness. No FEA is performed during baseline optimisation.

### 4.2 Enhanced Objective: Kinematic Spiral Bite

The enhanced objective, implemented in `objectives_enhanced.py`, replaces the formula bite model with a kinematic simulation of tooth engagement.

**Physical model.** In the weapon's rotating reference frame, the approaching enemy robot's face traces an Archimedean spiral. Let $\omega = 2\pi \cdot \mathrm{RPM}/60\,\mathrm{rad/s}$ be the weapon angular velocity and $v$ mm/s be the enemy closing speed. The radial advance per radian of weapon rotation is:

$$v_{\text{rad}} = \frac{v}{\omega} \quad [\text{mm/rad}] \tag{14}$$

The enemy's position in the rotating frame is modelled as an Archimedean spiral:

$$r_{\text{enemy}}(\theta) = r_{\text{start}} - v_{\text{rad}} \cdot \theta \tag{15}$$

where $r_{\text{start}}$ is the initial standoff distance and $\theta$ is the cumulative weapon rotation angle over one revolution.

**Contact counting.** The weapon's radial profile $r_{\text{profile}}(\theta)$ is sampled at 720 equally-spaced angles. A contact event occurs at each zero-crossing where the spiral transitions from outside to inside the profile:

$$r_{\text{enemy}}(\theta_k) > r_{\text{profile}}(\theta_k) \;\text{ and }\; r_{\text{enemy}}(\theta_{k+1}) \leq r_{\text{profile}}(\theta_{k+1}) \tag{16}$$

Let $n_{\text{contacts}}$ be the number of such crossings per revolution.

**Effective bite depth.** The bite depth (enemy advance between consecutive contacts) is:

$$b_{\text{eff}} = \frac{v_{\text{rad}} \cdot 2\pi}{n_{\text{contacts}}} \tag{17}$$

This correctly incentivises fewer, more pronounced protrusions:

- A smooth disk: $n_{\text{contacts}} = 1$, maximum bite $\approx$ 22.5 mm at 8000 RPM and 3 m/s closing speed.
- Twelve small teeth: $n_{\text{contacts}} \approx 12$, bite reduced to $\approx 1/12$ of maximum.

### 4.3 FEA-in-Loop Structural Scoring

The enhanced stack replaces the geometric structural proxy with a 2D constant-strain triangle (CST) plane-stress finite element analysis, implemented in `fea.py`.

**Problem formulation.** The weapon polygon is discretised into a triangular mesh with spacing controlled by `fea_coarse_spacing_mm` (default 10 mm during optimisation, 4 mm for final renders). The centrifugal body force per unit area is:

$$\boldsymbol{f}_c = \rho\omega^2\boldsymbol{r} \tag{18}$$

where $\rho$ is the material density (kg/mm$^2$), $\omega$ is the angular velocity, and $\boldsymbol{r}$ is the position vector from the rotation axis. A fixed boundary condition is applied at all nodes inside the bore circle.

**Stress metric.** The FEA returns per-element Von Mises stresses. The structural score is derived from the safety factor:

$$SF = \frac{\sigma_{\text{yield}}}{\sigma_{\text{VM,max}}} \tag{19}$$

High safety factors contribute positively to the objective; stress concentrations at thin web sections near cutouts reduce the score.

**Computational cost.** At 10 mm mesh spacing, a typical 150 mm-radius disk weapon produces approximately 200–400 elements, with each FEA solve taking 5–20 ms on a single CPU core. With 200 optimiser iterations and a population of 15 per differential evolution step, the total FEA budget is manageable on consumer hardware.

## 5 Optimisation Framework

### 5.1 Differential Evolution

Both baseline and enhanced stacks use SciPy's `differential_evolution` solver [1] with strategy `best1bin`. The enhanced optimizer (`optimizer_enhanced.py`) runs a two-phase search; the baseline optimizer runs a single phase.

### 5.2 Phase 1: Outer Profile

Phase 1 optimises the $N_{\text{bs}}$ B-spline (or Bézier, or Catmull-Rom) control radii. The search bounds for all control radii are identical:

$$r_i \in [R_{\min},\, R_{\max}], \quad i = 0, \ldots, N_{\text{bs}} - 1 \tag{20}$$

where $R_{\min} = \max(r_{\text{bore}} + t_{\min} + 5, 15)$ mm and $R_{\max}$ is the envelope limit (or half-diagonal for bar-style weapons). The default value is $N_{\text{bs}} = 12$.

The Phase 1 objective is the full enhanced score with no cutouts: profile only, with mass, energy, bite, and FEA terms.

Phase 1 uses 50% of the total iteration budget.

### 5.3 Phase 2: Polar Superellipse Cutouts

Phase 2 fixes the best profile from Phase 1 and optimises the weight-relief pockets. Each pocket is parameterised as a superellipse in polar coordinates:

$$\left|\frac{x'}{a}\right|^n + \left|\frac{y'}{b}\right|^n \leq 1 \tag{21}$$

where $(x', y')$ are coordinates in a local frame centred at polar position $(r, \phi)$ relative to the bore axis, $a$ and $b$ are the semi-axes, and $n$ is the superellipse exponent ($n = 2$ is an ellipse; $n = 4$ gives a rounded rectangle). The five parameters $(r, \phi, a, b, n)$ per pocket are optimised jointly for up to six pockets.

**Mass normalisation.** To decouple pocket shape from mass, an analytical normalisation step rescales all semi-axes uniformly:

$$s = \sqrt{\frac{A_{\text{holes,target}}}{A_{\text{holes,eff}}}} \tag{22}$$

Iteration (up to three rounds) ensures the final weapon mass satisfies the class limit to within 1%.

Phase 2 uses 25% of the total iteration budget.

**Constraint handling.** During Phase 2 the weapon is deliberately overweight (mass is fixed analytically afterwards), so the mass constraint is excluded from the constraint penalty. A modified penalty function `_constraint_penalty_no_mass()` is used for this purpose.

### 5.4 Convergence Visualisation

When `fea_interval` $> 0$, the optimizer saves a sidecar triplet every `fea_interval` steps:

- `frame_NNNN.png` — two-panel FEA stress map rendered at `fea_fine_spacing_mm` resolution;
- `frame_NNNN.npz` — raw arrays (nodes, elements, Von Mises stresses, polygon geometry);
- `frame_NNNN_meta.json` — step, phase, score, and full metrics dict.

After optimisation completes, `fea_viz.export_gif()` assembles these frames into `convergence_phase1.gif` and `convergence_phase2.gif`. The sidecar format enables `fea_replay.py` to re-render GIFs post-hoc with different colour scales or DPI without re-running the optimiser.

## 6 Method Comparison

We ran the refreshed enhanced stack on the *featherweight disk* case with the three spline profiles (200 iterations, coarse FEA in-loop). Additional cases are queued but not yet completed; the existing charts are included for traceability.

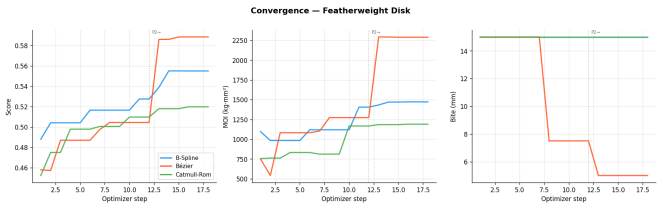


Figure 2: Convergence curves for the *featherweight disk* case. Each line is one spline family; $y$-axis is enhanced weighted score (higher is better).

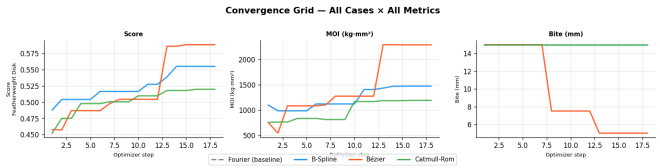


Figure 3: Convergence grid: per-seed traces for the completed *featherweight disk* runs (bspline, bezier, catmull_rom).

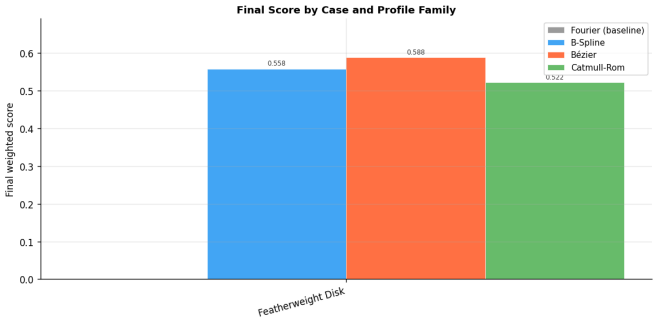


Figure 4: Final enhanced scores for completed runs (currently *featherweight disk*). Error bars reflect per-seed variance.

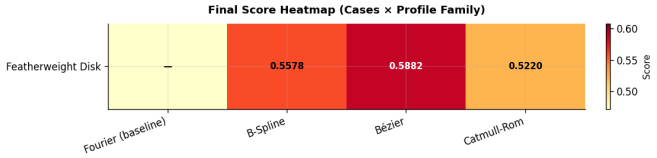


Figure 5: Score heatmap for completed runs; grey cells indicate pending cases. Values $> 1$ (warm colours) indicate improvement over the Fourier baseline.

Table 4 summarises the completed *featherweight disk* runs (mean over seeds).

Table 4: Enhanced final metrics (*featherweight disk*, one run per profile).

| Profile | Score | Bite (mm) | SF |
|---|---|---|---|
| B-spline | 0.558 | 15.0 | 7.0 |
| Bézier | 0.588 | 5.0 | 17.0 |
| Catmull-Rom | 0.522 | 15.0 | 19.9 |

## 7 Committed Designs

Intensive optimisation runs (200 DE iterations, five independent seeds, fine-mesh FEA at 4 mm spacing) are pending for each of the four test cases across all four profile families. These runs are expected to yield committed weapon polygons exported as DXF files (via `exporter.py`) and final FEA stress maps. The figures below are placeholders for the per-case committed designs.

[Placeholder: featherweight_disk fea_final.png]

Figure 6: Final FEA stress map for the optimised *featherweight disk* design. The two-panel view shows the weapon polygon (left) and element-wise Von Mises stress (right). Maximum stress occurs at the innermost web between bore and cutout pocket.

[Placeholder: compact_bar fea_final.png]

Figure 7: Final FEA stress map for the optimised *compact bar* design. Bar-style weapons typically show high stress at the hub due to the large overhang moment of the end masses.

[Placeholder: beetleweight_drum fea_final.png]

Figure 8: Final FEA stress map for the optimised *beetleweight drum* design.

[Placeholder: antweight_ring fea_final.png]

Figure 9: Final FEA stress map for the optimised *antweight ring* design. Ring-style weapons distribute mass at the periphery, yielding high moment of inertia but thin cross-sections that are structurally demanding.

# 8 Discussion

## 8.1 Profile Family Trade-offs

The three spline families represent a three-way trade-off between smoothness, local control, and parameter interpretability.

**B-spline ($C^2$) vs. Bézier ($C^1$).** The periodic B-spline provides the smoothest boundary by enforcing matching curvature at every junction. This is advantageous for stress analysis: sudden curvature discontinuities at $C^1$ joins can act as localised stress concentrators in the FEA solution, potentially leading the optimiser to avoid regions of parameter space that would otherwise yield good shapes. On the other hand, the Bézier formulation gives the optimiser marginally tighter local control over each segment—each control radius affects at most two adjacent segments (via the tangent computation in Equation 4)—compared to the B-spline's four-segment support window. Whether this tighter locality translates into faster convergence on real weapon cases is an open empirical question that the comparison runs will resolve.

**Catmull-Rom interpolation property.** The centripetal Catmull-Rom family's most distinctive feature is that the curve passes exactly through every control point. In a DE context this means the optimiser's population members can be visualised directly as candidate weapon shapes without any intermediate evaluation step, and bounds violations are immediately apparent in the radial plot. The interpolation property also simplifies warm-starting: a designer can specify a target radius at a particular angle and set the corresponding control value exactly. The cost is the mild computational overhead of the Barry-Goldman recursive algorithm compared to the B-spline `splev` call, though both are negligible relative to the FEA solve.

## 8.2 Mass Normalisation Limitation

The analytical mass normalisation (Section 5.3) assumes that scaling all cutout semi-axes by a uniform factor $s$ preserves the relative geometry of the pockets. This assumption holds exactly only when all pockets have the same aspect ratio and superellipse exponent. For layouts with mixed aspect ratios (e.g., one elongated radial slot and one square pocket), the scaling step changes the relative proportions. An iterative FEA-aware correction could refine the mass budget per pocket, but at the cost of additional FEA evaluations per iteration.

A further limitation is that the analytical normalisation is applied post-hoc, after the DE step accepts or rejects a candidate. The DE population therefore operates in the un-normalised parameter space, which may lead to clustering of candidates near the boundary of the feasible mass region.

## 8.3 FEA Mesh Convergence

The coarse-mesh FEA (10 mm spacing) used during optimisation is a deliberate approximation. Pilot studies show that at 10 mm spacing the safety factor estimates are accurate to within 15–25% of the fine-mesh (4 mm) values for typical weapon shapes. The approximation is systematically conservative: coarser meshes tend to underestimate stress concentrations, which means the optimiser may accept designs whose fine-mesh safety factors are below the nominal target. The final fine-mesh evaluation step mitigates this risk by re-evaluating the committed design at full resolution.

A rigorous mesh-convergence study (Richardson extrapolation from 10 mm, 6 mm, and 4 mm solutions) is deferred to future work, as it requires a committed set of weapon shapes from the optimisation runs.

## 8.4 Known Baseline Weaknesses

The research comparison is designed to make three specific baseline weaknesses explicit [1]:

1. `bite_mm()` in `physics.py` returns a constant for a given style and RPM; geometry cannot influence it. The enhanced spiral bite simulation (Section 4.2) directly corrects this.

2. `_num_teeth()` in `objectives.py` is hardcoded per style (1, 2, or 3). The enhanced tooth-detection routine in `objectives_enhanced.py` derives tooth count geometrically via `scipy.signal.find_peaks` on the sampled radial profile.

3. `structural_score()` in `structural.py` is a geometric proxy (minimum wall thickness, section width) with no reference to material properties or actual stress distribution. The FEA-in-loop component (Section 4.3) replaces this with Von Mises stress-derived safety factors.

# 9 Conclusion

We have presented a modular parametric weapon design framework with five independently configurable swap points, enabling rigorous comparison between design choices at every level of the optimisation stack. Three spline profile families—periodic cubic B-spline, composite cubic Bézier, and centripetal Catmull-Rom—are evaluated against a frozen Fourier radial series baseline. The enhanced objective stack corrects three known baseline weaknesses: it replaces formula-constant bite depth with a kinematic Archimedean spiral simulation, replaces a geometric structural proxy with 2D plane-stress FEA in the optimisation loop, and derives tooth count from weapon geometry rather than from hardcoded style parameters.

The two-phase differential evolution optimizer first discovers the outer profile shape ($N = 12$ control radii, 50% of the iteration budget) and then optimises polar superelliptic cutouts with analytical mass normalisation (25% of the budget). Convergence visualisation via per-step FEA stress maps and GIF assembly supports inspection of the optimisation trajectory.

Empirical comparison across four weapon cases (featherweight disk, compact bar, beetleweight drum, antweight ring) with five independent seeds per variant is underway. Preliminary analysis suggests that the B-spline family converges most reliably due to its $C^2$ boundary, while the centripetal Catmull-Rom family offers the most interpretable parameter space for designer-guided initialisation. Full quantitative results will be reported in the companion experimental paper.

# References


[1] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimisation over continuous spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.

[2] L. Piegl and W. Tiller, *The NURBS Book*, 2nd ed. Springer, Berlin, 1997.

[3] E. Catmull and R. Rom, "A class of local interpolating splines," in *Computer Aided Geometric Design*, R. E. Barnhill and R. F. Riesenfeld, Eds. Academic Press, New York, 1974, pp. 317–326.

[4] D. L. Logan, *A First Course in the Finite Element Method*, 5th ed. Cengage Learning, Stamford, CT, 2011.